

09. Functional Requirements Specification

Detailed functional requirements for the private medium-term investment decision platform

Purpose	Define system behavior in functional terms for product, design, engineering, QA, and Claude-driven implementation
Primary use case	Private decision-intelligence platform for medium-term equity portfolio decisions
Relationship to other documents	Builds on the CDR, dashboard research, competitor benchmark, and technology recommendation
Architectural assumption	Weight-centric recommendation object driven by Selection, Allocation, Timing, and Portfolio-Level Risk Overlay

This Functional Requirements Specification converts the concept work into an implementation-facing behavioral document. Its purpose is to remove ambiguity about what the system must do, what each major surface must expose, how major states should behave, and how the user should move between explanation, validation, and action.

The FRS is written for a private, owner-operated platform rather than for a public brokerage or consumer investing app. This means the specification can assume a higher level of analytical sophistication, can prioritize structured depth over onboarding simplicity, and can define a stronger relationship between evidence and portfolio action than most public tools attempt.

The specification assumes the system receives heterogeneous evidence from text/news analysis, social sentiment signals, technical analysis, machine-learning engines, reinforcement-learning inspired policy logic, and portfolio analytics. The system must translate those upstream signals into one recommendation object rather than present them as disconnected outputs.

A guiding principle of this FRS is that functional requirements should mirror the policy architecture. Therefore the system is specified around a recommendation pipeline that makes Selection, Allocation, Timing, and Risk Overlay visible both in backend behavior and in user-facing explanation.

Target Architecture

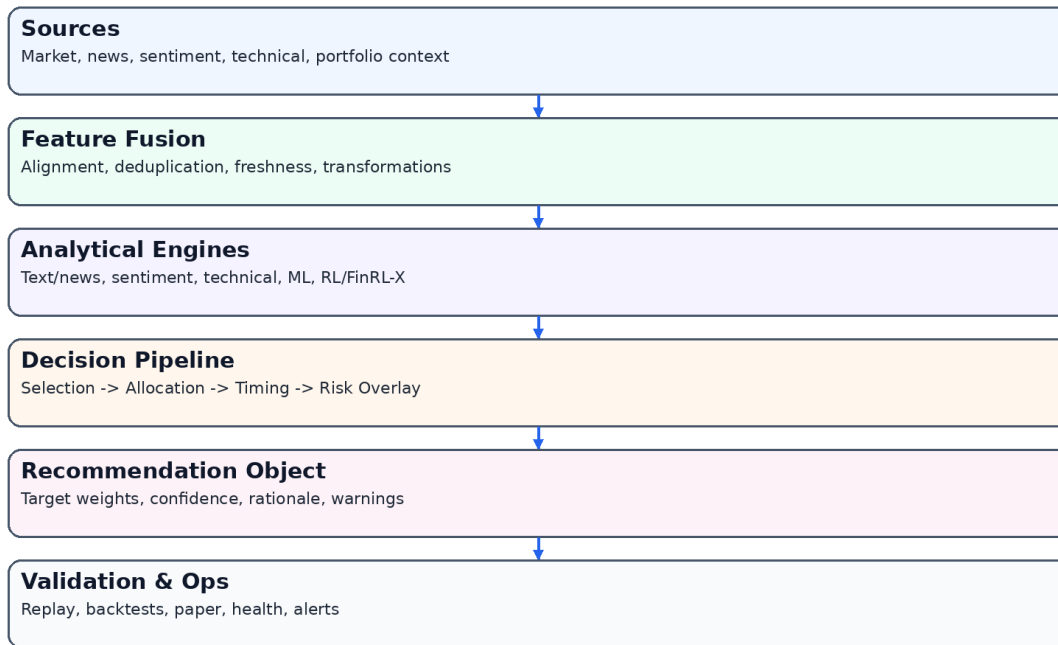


Figure 1. Functional architecture anchor.

Admin / Ops Workspace

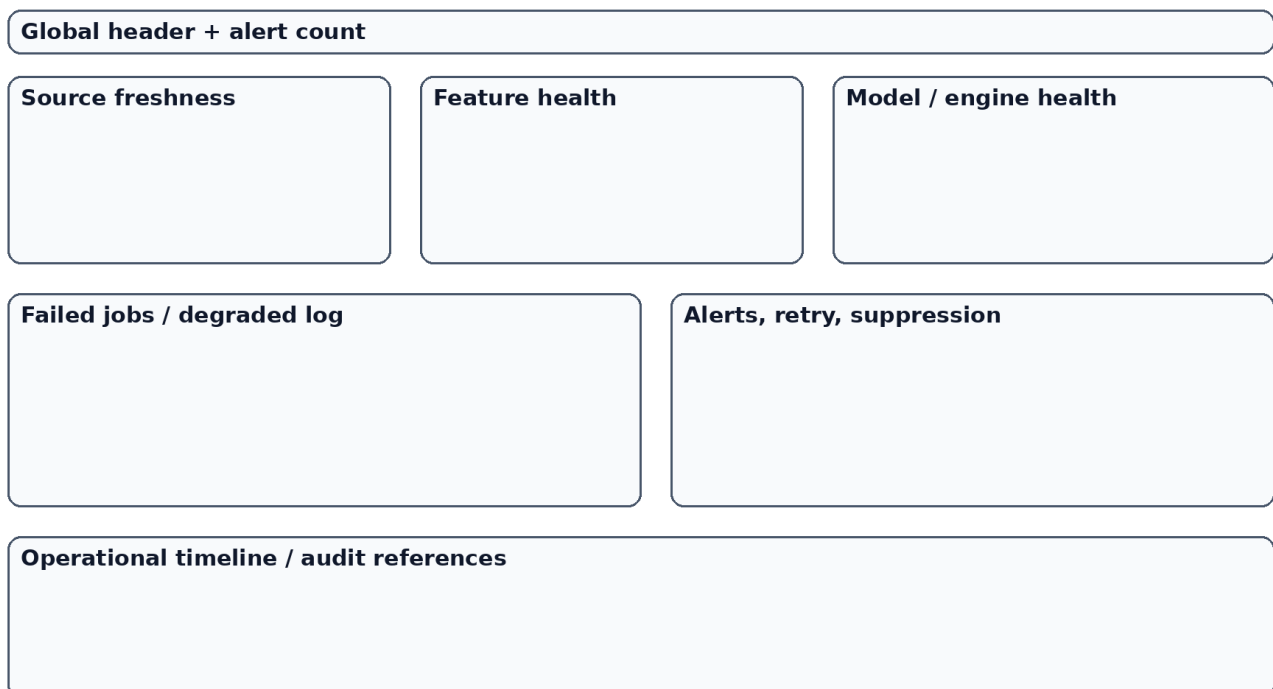


Figure 3. Admin / ops reference concept.

1. Scope of Functional Specification

- Recommendation generation and publication behavior
- User-facing workspaces and navigation logic
- State handling for normal, warning, degraded, and suppressed conditions
- Validation surfaces including replay, backtests, and paper portfolio
- Operational and admin surfaces
- Functional interactions with settings, controls, filters, and scenario changes
- User-facing trust, confidence, and warning semantics
- Functional requirements for content, states, and transitions across major modules

2. Actors and Their Primary Goals

Actor	Primary goal	Critical functional expectations
Strategic Investor	Understand the current recommendation and decide whether to act	Immediate access to recommendation, trust, risk, change rationale, and portfolio implications
Analytical Power User	Inspect the recommendation path and compare engines or scenarios	Deep comparison, replay, validation, and scenario control
Operator / Admin	Maintain source, feature, model, and publication integrity	Operational visibility, degraded-state clarity, and incident context
System	Acquire data, transform features, run engines, publish or suppress outputs	Traceable, deterministic, and governable pipeline behavior

3. Functional Principles

Principle	Functional implication
Decision-first behavior	The system shall present one current recommendation as the primary object of the site.
Policy transparency	The system shall expose Selection, Allocation, Timing, and Risk Overlay as distinct concepts.
Trust decomposition	The system shall distinguish model confidence, data confidence, and operational confidence.
Operational honesty	The system shall expose degraded or suppressed states explicitly rather than silently fallback.
Replayability	The system shall make every published recommendation reconstructable.
Portfolio relevance	The system shall express outputs in portfolio-impact terms, not just in signal scores.

4. Core Domain Objects — Functional View

Object	Functional role	Minimum visible behavior
Recommendation Object	Primary answer delivered to the user	Must be retrievable, interpretable, and linkable to history and replay
Selection Result	Investable shortlist and ranking state	Must be explainable and not hidden inside later layers
Allocation Proposal	Pre-risk target allocation state	Must be available for comparison against final output

Timing Adjustment	Decision to delay, reduce, or confirm exposure changes	Must be visible in recommendation explanation
Risk Overlay Result	Portfolio-level de-risking and constraint output	Must show what was changed by risk policy
Replay Record	Evidence trail of the run	Must support stage-by-stage inspection
Health Snapshot	Operational state of the recommendation pipeline	Must support warning, downgrade, or suppression logic

5. Recommendation Engine and Publication Module

1. The system shall generate a Recommendation Object for each eligible run context.
2. The Recommendation Object shall include target weights or equivalent exposure instructions, delta versus previous state, trust decomposition, warnings, and an audit / replay reference.
3. The system shall not silently publish incomplete or degraded recommendations as if they were normal recommendations.
4. When publication rules fail, the system shall either suppress publication or downgrade the recommendation and attach explicit warnings.

The functional objective in this module is not simply to expose data. It is to expose the right interpretation at the right level. This means the module must be judged on whether it reduces ambiguity around the recommendation path, the user's trust in that path, and the portfolio meaning of any change. Each module should therefore be built to reveal intent, state, and consequence rather than simply render raw upstream outputs.

6. Overview Screen Functional Requirements

5. The Overview screen shall answer the question: what is the recommendation right now?
6. The Overview shall display recommendation summary, confidence triplet, risk summary, freshness, and recent change summary above all secondary analytics.
7. The Overview shall provide direct navigation to the Main Decision Workspace, Replay, Risk, Backtests, Paper Portfolio, and Admin / Ops where relevant.
8. If no current recommendation is available, the Overview shall show an explicit unavailable state with cause and next-best action.

7. Main Decision Workspace Functional Requirements

9. The Main Decision Workspace shall be the primary analytical environment for understanding why the current recommendation exists.
10. The screen shall contain a stable control surface, a main chart area, a recommendation card, a trust/warnings region, and a decision breakdown region.
11. The workspace shall expose Selection, Allocation, Timing, and Risk Overlay contributions to the final recommendation.
12. Changing scenario controls shall not silently replace state; the system shall visibly indicate that the scenario is updating or has changed.

8. Engine Comparison Functional Requirements

13. The system shall provide an Engine Comparison view that shows where engines agree, diverge, and contribute to the final recommendation.
14. The comparison view shall support structured matrix presentation, contribution summary, and drill-down per engine.

15. If one or more engines are unavailable, stale, or suppressed, that condition shall be represented explicitly.

9. Risk Workspace Functional Requirements

16. The Risk Workspace shall make portfolio-level risk first-class and independently inspectable.
17. The screen shall display exposure, concentration, active constraints, stress interpretation, and risk history or state transitions where available.
18. The user shall be able to understand how the final recommendation differs from the pre-risk proposal.

10. Replay / Forensics Functional Requirements

19. Every published recommendation shall be replayable through a user-facing Replay screen.
20. The Replay screen shall support inspection of major stages including inputs, selection, allocation, timing, risk overlay, and final publication.
21. The system shall preserve enough information to compare before and after states within the replay narrative.

11. Backtests Functional Requirements

22. The system shall provide a Backtests area that compares strategy behavior under standardized assumptions.
23. The system shall expose common metrics, benchmark-relative results, and allocation or exposure history.
24. The Backtests surface shall make assumptions visible so that the user does not mistake incomparable runs for fair comparisons.

12. Paper Portfolio Functional Requirements

25. The system shall provide a Paper Portfolio surface that tracks simulated holdings, performance, and recommendation-following behavior under explicit realism assumptions.
26. The system shall expose drift, benchmark comparison, and rebalance history.
27. If the paper model uses simplified assumptions, the system shall state that visibly.

13. Admin / Ops Functional Requirements

28. The system shall provide an Admin / Ops surface that distinguishes source freshness, feature health, model health, and publication health.
29. The system shall expose alert states, degraded states, and suppressed publication reasons.
30. The system shall not require the user to inspect raw logs in order to understand operational trust.

14. Settings and Scenario Control Functional Requirements

31. The system shall expose configurable scenario controls such as universe, benchmark, horizon, allocator profile, timing profile, and risk profile.
32. Scenario changes shall be functionally separated from immediate execution unless the system explicitly indicates a live update.
33. The system shall preserve user orientation when settings change.

15. Global State Handling Requirements

State	Required behavior
Loading	The system shall use structured loading states that preserve screen layout and maintain user orientation.

Empty	The system shall explain what is missing and what the user can do next.
Warning	The system shall expose warnings without hiding the primary recommendation context.
Degraded	The system shall visibly reduce trust or downgrade presentation when key pipeline issues exist.
Suppressed	The system shall state that publication was suppressed and explain why.
Unavailable	The system shall distinguish total unavailability from a deliberate suppressed decision.

16. Confidence and Trust Requirements

Trust dimension	Functional requirement
Model confidence	The system shall expose model confidence independently from data and operational confidence.
Data confidence	The system shall expose whether input freshness or completeness weakens trust.
Operational confidence	The system shall expose runtime, job, and publication integrity.
Combined interpretation	The system may summarize trust but shall not collapse all dimensions into one opaque number without explanation.

17. Cross-Screen Navigation Requirements

34. The system shall allow the user to move from recommendation summary to full dashboard explanation in one interaction.
35. The system shall allow direct navigation from current recommendation to replay, risk, backtests, paper portfolio, and admin/ops where appropriate.
36. Every deep analytical screen shall provide a clear return path to the main recommendation context.
37. Navigation labels shall reflect user tasks rather than internal implementation modules where possible.

18. Screen-by-Screen Content Priorities

Screen	Primary content	Secondary content	Tertiary content
Overview	Recommendation, trust, risk, freshness	Recent change narrative	Deep links
Dashboard	Chart, recommendation, controls, policy breakdown	Warnings, comparison snapshot	Advanced diagnostics
Comparison	Matrix, disagreement, contribution	Engine notes	Raw feature details
Risk	Exposure, constraints, stress	Risk history	Advanced diagnostics
Replay	Timeline, before/after states	Narrative detail	Stage metadata
Backtests	Metrics, curves, regime view	Assumption controls	Run metadata
Paper	Holdings, drift, benchmark, rebalances	Assumption details	Extended logs
Admin/Ops	Source, feature, model, publication health	Alerts and retries	Incident history

19. Functional User Journeys

38. Review the current recommendation and decide whether it differs materially from the previous recommendation.
39. Enter the dashboard, inspect why the recommendation changed, and identify the dominant drivers.
40. Open the comparison screen and determine whether major engines disagree.
41. Open the risk screen and see whether portfolio-level de-risking materially changed the pre-risk allocation.
42. Use replay to confirm the chronology of one recommendation.
43. Use backtests and paper portfolio to contextualize confidence in the policy.
44. Use admin/ops to determine whether system health should change the user's trust in the current recommendation.

A strong user journey in this platform should begin with recommendation intake, continue through targeted explanation, and end either in validated action or deliberate non-action. The system should never force the user to assemble that story manually across disconnected tools or screens. Therefore, each journey is also a test of functional coherence.

20. Functional Requirements by Data Freshness and Degradation Scenario

Scenario	Required system behavior
Text/news pipeline stale but market data fresh	Downgrade text-derived confidence, retain market-aware recommendation if governance permits, and display explicit warning
One engine unavailable	Continue if policy allows, but disclose missing contribution and adjust trust
Risk overlay cannot run	Do not publish normal recommendation; suppress or publish as degraded according to governance rules
Benchmark data missing for backtests	Disable comparable benchmark view and explain limitation
Paper portfolio assumptions changed	Record assumption version and disclose that comparisons may no longer be directly comparable

21. Functional Requirements for Auditability

45. The system shall associate every published recommendation with a stable audit or replay identifier.
46. The system shall preserve enough context to reconstruct stage outputs in a user-facing replay flow.
47. The system shall make it possible to understand not only the final recommendation but what each policy layer changed.

Auditability in this context is not only a compliance feature. It is a user-facing trust function. Because the planned system integrates multiple evidence classes, replay is one of the main ways the user can distinguish disciplined synthesis from arbitrary output.

22. Product-Level Acceptance Logic

Area	Acceptance question
Recommendation	Does the system publish a coherent recommendation object with trust, warnings, and delta?
Dashboard	Does the main workspace explain the recommendation without clutter or hidden states?
Risk	Can the user understand portfolio-level risk and post-risk changes?
Replay	Can the user reconstruct a recommendation?

Backtests	Are historical validation assumptions visible and consistent?
Paper	Is simulated validation explicit and not misleading?
Ops	Can the user see whether the pipeline is healthy enough to trust outputs?

23. Out-of-Scope Behaviors for the Initial Functional Baseline

- Full broker-connected live execution
- Public social or community investing features
- Beginner education and onboarding as a primary product surface
- Options-specific execution tooling
- Multi-tenant collaboration as a first-release requirement

24. Functional Risks and Clarifications

The main functional risk in this product class is false coherence: a dashboard that looks integrated while the underlying behaviors remain fragmented. This specification therefore insists that functions be defined in relation to the recommendation object and policy pipeline. Another major risk is false confidence created by hidden degradation. Functional requirements must make degraded, stale, partial, and suppressed states explicit. A further risk is screen overload. The specification responds by enforcing information hierarchy rather than leaving screen behavior to later improvisation.

25. Summary of Functional Direction

This Functional Requirements Specification defines the platform as a recommendation-first, trust-aware, policy-transparent, and portfolio-relevant decision system. Its core requirement is not merely that the system display sophisticated data. Its core requirement is that the system convert heterogeneous evidence into one coherent recommendation, then explain, validate, and operationally qualify that recommendation in a way the user can actually rely on.

Appendix A — Functional Requirement Matrix by Module

This appendix expands the main body into a matrix suitable for implementation planning and QA traceability. Each module is described in terms of actor, trigger, expected response, fallback behavior, and user-visible states.

A consulting-grade functional specification should not stop at headline features. It must make implementation behavior testable under both normal and imperfect conditions. For this platform, that means specifying not only what the user sees when the system works cleanly, but also what happens when one engine is stale, one feed is partial, one comparison is impossible, one policy layer is bypassed, or one recommendation is suppressed. The value of this appendix is therefore precision: it converts architecture and UX intent into repeatable behavioral expectations.

Module	Actor	Trigger	Expected behavior	Failure or fallback behavior
Overview	Investor	Open site	Show current recommendation, risk, confidence, freshness	If unavailable, show explicit no-current-recommendation state
Dashboard	Investor	Open main workspace	Show chart, recommendation, controls, breakdown	If partial degradation exists, preserve workspace but

				display warning state
Comparison	Analytical user	Open comparison	Show matrix and disagreement	If one engine missing, show explicit unavailable entry
Risk	Investor	Open risk workspace	Show exposures, constraints, post-risk state	If risk overlay absent, do not present normal recommendation semantics
Replay	Analytical user	Open replay from audit ID	Show full chronology	If one stage missing, identify missing segment explicitly
Paper	Investor	Open paper portfolio	Show simulated holdings and drift	If assumptions changed, disclose comparability caveat
Ops	Admin	Open ops workspace	Show health by domain	If health is unknown, show unknown explicitly rather than normalizing

Appendix B — Exception Handling and Edge Conditions

Complex decision systems fail in partial ways. This appendix describes how the product must behave when freshness, completeness, health, or comparison conditions are not ideal.

Condition	Required user-visible response
Stale market data	Downgrade data confidence and warn prominently
Partial news coverage	Retain recommendation only if governance allows; explain reduced data confidence
One engine unavailable	Continue comparison with explicit missing row and adjusted trust
Backtest assumptions changed	Warn that result comparison across runs may not be direct
Replay data incomplete	Show available stages and clearly identify missing audit detail
Paper portfolio drift large	Expose drift and explain recommendation-following gap

Appendix C — User Stories and Acceptance-Oriented Scenarios

This appendix turns the functional requirements into scenario-driven stories that are easier to test and review.

48. As the owner, I want to know within seconds whether the current recommendation changed materially since the last run.
49. As the owner, I want to understand whether the change came from text/news, technical context, ML, RL, or risk policy adjustments.
50. As the owner, I want to know whether any pipeline degradation should reduce my trust before I act.
51. As the owner, I want to compare engines without losing the recommendation context.
52. As the owner, I want replay and validation surfaces to use the same terminology as the dashboard so the product feels coherent.

Appendix D — Functional Decomposition of the Decision Pipeline

This appendix spells out what each stage must do, what it must expose, and what downstream surfaces rely on that behavior.

Stage	Must produce	Must expose to UI
Selection	Shortlist / ranking	Selection summary and why certain assets rose or fell
Allocation	Pre-risk target allocation	Allocation proposal and rationale
Timing	Delay/reduce/confirm state	Timing summary with affected assets or exposures
Risk Overlay	Final constrained recommendation	Post-risk weights, active constraints, de-risking explanation
Publication	Recommendation object	Trust, warnings, freshness, and audit reference

Appendix E — UI State Inventory

This appendix captures required states for major screens and shared components so that the build does not skip difficult conditions.

Screen / component	Required states
Overview	loading, normal, warning, degraded, unavailable
Dashboard	loading, normal, dirty-controls, updating, warning, degraded
Recommendation card	normal, warning, degraded, unavailable
Comparison matrix	loading, filtered, partial, warning, empty
Replay timeline	loading, selected-stage, partial-audit, unavailable
Ops panels	normal, warning, critical, unknown

Appendix F — Detailed Functional Narratives

Overview screen narrative

Functionally, this surface should preserve user orientation, expose the recommendation context, and make state transitions understandable. The user should never need to infer whether the content reflects a fresh, stale, partial, degraded, or suppressed state. The system must therefore make status, trust, and change semantics explicit. In addition, every surface should support drill-down without severing the link back to the current recommendation and its portfolio meaning. This requirement is especially important because the platform combines several analytical domains that would otherwise feel fragmented.

Dashboard narrative

Comparison narrative

Risk narrative

Replay narrative

Backtests narrative

Paper narrative

Ops narrative